

```
#include <Servo.h>
```

```
// ----- L298N MOTOR DRIVER PINS -----
```

```
// LEFT side motors (both left motors wired in parallel to OUT1/OUT2)
```

```
#define ENA 5 // PWM speed control - Left side
```

```
#define IN1 8
```

```
#define IN2 7
```

```
// RIGHT side motors (both right motors wired in parallel to OUT3/OUT4)
```

```
#define ENB 6 // PWM speed control - Right side
```

```
#define IN3 4
```

```
#define IN4 12
```

```
// ----- ULTRASONIC SENSOR (HC-SR04) -----
```

```
#define TRIG_PIN A1
```

```
#define ECHO_PIN A0
```

```
// ----- IR SENSORS -----
```

```
#define IR_LEFT A2
```

```
#define IR_RIGHT A3
```

```
// NOTE: Most IR obstacle modules output LOW when they DETECT an object.
```

```
// If your robot's behavior seems backwards, flip IR_ACTIVE_STATE below.
```

```
#define IR_ACTIVE_STATE LOW
```

```
// ----- SERVO (pans ultrasonic sensor) -----
```

```
#define SERVO_PIN 10
```

```
Servo panServo;

int servoCenter = 90; // straight ahead

int servoLeft  = 150;

int servoRight = 30;


// ----- TUNING VALUES -----

int STOP_DISTANCE  = 15; // cm - too close, stop/back up
int FOLLOW_DISTANCE = 40; // cm - max distance to still "see" the person
int BASE_SPEED     = 160; // 0-255 forward speed
int TURN_SPEED     = 140; // 0-255 speed while steering


unsigned long lastSeenTime = 0;

const unsigned long SEARCH_DELAY = 1500; // ms with nothing seen before searching


void setup() {

  Serial.begin(9600);


  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);


  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
```

```
pinMode(IR_LEFT, INPUT);
```

```
pinMode(IR_RIGHT, INPUT);
```

```
panServo.attach(SERVO_PIN);
```

```
panServo.write(servoCenter);
```

```
stopMotors();
```

```
}
```

```
void loop() {
```

```
    long distance = getDistanceCM();
```

```
    bool leftDetect = (digitalRead(IR_LEFT) == IR_ACTIVE_STATE);
```

```
    bool rightDetect = (digitalRead(IR_RIGHT) == IR_ACTIVE_STATE);
```

```
    Serial.print("Dist: "); Serial.print(distance);
```

```
    Serial.print(" cm | L:"); Serial.print(leftDetect);
```

```
    Serial.print(" R:"); Serial.println(rightDetect);
```

```
    bool personDetected = (distance > 0 && distance <= FOLLOW_DISTANCE);
```

```
    if (personDetected) {
```

```
        lastSeenTime = millis();
```

```
        panServo.write(servoCenter); // keep sensor facing forward while tracking
```

```
        if (distance <= STOP_DISTANCE) {
```

```

    // Close enough - stop right in front of the person
    stopMotors();
}
else if (leftDetect && !rightDetect) {
    turnLeft();
}
else if (rightDetect && !leftDetect) {
    turnRight();
}
else {
    moveForward();
}
}
else {
    stopMotors();
    // If nothing seen for a while, sweep the servo to search
    if (millis() - lastSeenTime > SEARCH_DELAY) {
        searchForPerson();
    }
}

delay(60);
}

// ===== MOVEMENT FUNCTIONS =====

void moveForward() {

```

```
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  
digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);  
analogWrite(ENA, BASE_SPEED);  
analogWrite(ENB, BASE_SPEED);  
}
```

```
void turnLeft() {  
    // Right side moves faster/forward, left side slows/reverses slightly -> pivot left  
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);  
    analogWrite(ENA, TURN_SPEED);  
    analogWrite(ENB, TURN_SPEED);  
}
```

```
void turnRight() {  
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);  
    analogWrite(ENA, TURN_SPEED);  
    analogWrite(ENB, TURN_SPEED);  
}
```

```
void stopMotors() {  
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);  
    analogWrite(ENA, 0);  
    analogWrite(ENB, 0);  
}
```

```
}
```

```
// ===== SENSOR FUNCTIONS =====
```

```
long getDistanceCM() {
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
    delayMicroseconds(2);
```

```
    digitalWrite(TRIG_PIN, HIGH);
```

```
    delayMicroseconds(10);
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
    long duration = pulseIn(ECHO_PIN, HIGH, 25000); // 25ms timeout (~4m range)
```

```
    if (duration == 0) return -1; // no echo received
```

```
    long distance = duration * 0.0343 / 2; // convert to cm
```

```
    return distance;
```

```
}
```

```
void searchForPerson() {
```

```
    // Simple left-right sweep to try to spot the person again
```

```
    panServo.write(servoLeft);
```

```
    delay(400);
```

```
    long distLeft = getDistanceCM();
```

```
    if (distLeft > 0 && distLeft <= FOLLOW_DISTANCE) {
```

```
        lastSeenTime = millis();
```

```
        panServo.write(servoCenter);
```

```
        return;
```

```
}
```

```
panServo.write(servoRight);
```

```
delay(400);
```

```
long distRight = getDistanceCM();
```

```
if (distRight > 0 && distRight <= FOLLOW_DISTANCE) {
```

```
    lastSeenTime = millis();
```

```
    panServo.write(servoCenter);
```

```
    return;
```

```
}
```

```
panServo.write(servoCenter);
```

```
}
```